Mathy Resurrected

v0.0.2 README

1. Numeric input

All numbers are treated as complex numbers. It is allowed to use either 'i' or 'j' to write complex unit $(\sqrt{-1})$. Assuming '.' as decimal separator following are examples of supported numeric input:

Input	interpreted as
12	12+0i
12e12	12*10^12+0i
12E+12	12*10^12+0i
12e-12	12*10^(-12)+0i
12.12	12.12+0i
12.12e12	12.12*10^12+0i
.12	0.12+0i
-.12e-2	-0.12*10(-2)+0i
12+3i	12+3i
12-12e12+i	12.12*10^(-12)+i
i	0+i
-i	0-i
-12i	0-12i
...	...

Table 1: Examples of numeric input

It is also possible to enter percentage directly like in expressions : 2%, 3%i, 2+33%...

1.1. Binary, hexadecimal and octal numbers

All these bases are supported in input, but only as unsigned integers. If present in output (which is configurable) these numbers represent cut-off value of actual calculation which is done with multiple precision floats. When found in input, numbers must be prefixed with their base prefix:

- "0b" or "0B" - for binary numbers.
- "0x" or "0X" - for hexadecimal numbers.
- "0" - for octal numbers.

2. Operators

name	symbol	example
unary plus	+	+1, +2i
unary minus	-	-1, -3i

Table 2: Unary operators

name	symbol	example
plus/addition	+	1+3i
minus/substraction	-	2-5i
multiply	*	3*8
divide	/	4/35
the power of	^	2^4
modulo[1]	mod	5 mod 3

Table 3: Binary arithemtic operators

symbol(s)	description	usage
&; and	bitwise and	x & y; x and y
|; or	bitwise or	x & y; x or y
~; xor	bitwise xor	x ~ y; x xor y
!; not	bitwise not	!x; not x
>>; shr	bitwise shift right	x >> y; x shr y
<<; shl	bitwise shift left	x << y; x shl y
nand	bitwise nand	x nand y
nor	bitwise nor	x nor y
xnor	bitwise xnor	x xnor y

Table 4: Bitwise operators

Bitwise operator work only on real part of expression, and before they are applied whole expression is converted to signed integer type with one of configurable fixed bit-widths.

Operator precedence follows closely well known operator precedence from C/C++ and math in general. There are some operations that are usuallly not available as operators (nand, nor, xnor). For these, precedence had to be invented, so following table gives complete operator precedence in Mathy Resurrected.

1 Note that although all calculation are preformed on complex numbers, there are some operations that make no sense for them. In these cases, given operation either ignores imaginary part of operands or sets it to 0 and preforms operation only on real parts. Modulo operator is one of these operations.

operator	preccedence	associativity
unary+ unary- !	1	RTL
^	2	RTL
* / mod	3	LTR
+ -	4	LTR
shl shr	5	LTR
nand nor xnor	6	LTR
&	7	LTR
~	8	LTR
|	9	LTR

Table 5: Oprator precedence

Higher precedence means lower order of evaluation: if operator op1 has precedence 3 and operator op2 precedence 4, op1 will be evaluated before op2.

3. SI and binary unit prefixes

This is special functionality that allows you to write SI unit prefix as a number suffix and that number is automatically converted into base unit quantity:

2k = 2 * 1 0000 = 2.000

2M = 2 * 1 000 000 = 2.000.000

and also:

2Ki = 2 * 1024 = 2048

Note that you can't write prefix alone (as you can with imaginary unit):

k or 1*k

But number must be present and is implicitly multiplied:

1k

SI SUFFIXES ARE CASE SENSITIVE!

Quantity	Prefix	Symbol
10^24	yotta	Y
10^21	zetta	Z
10^18	exa	E
10^15	peta	P
10^12	tera	T
10^9	giga	G
10^6	mega	M
10^3	kilo	k
10^2	hecto	h
10^1	deca	da
10^-1	deci	d
10^-2	centi	c
10^-3	milli	m
10^-6	micro	u
10^-9	nano	n
10^-12	pico	p
10^-15	femto	f
10^-18	atto	a
10^-21	zepto	z
10^-24	yocto	y

Table 6: SI unit prefixes

Name	Symbol	Base 2	Base 1024
kibi	Ki	2^10	1024^1
mebi	Mi	2^20	1024^2
gibi	Gi	2^30	1024^3
tebi	Ti	2^40	1024^4
pebi	Pi	2^50	1024^5
exbi	Ei	2^60	1024^6
zebi	Zi	2^70	1024^7
yobi	Yi	2^80	1024^8

Table 7: Binary unit prefixes

4. Functions

MathyResurrected has a set of built in functions. Function arguments are separated by cofigurable separator. Functions are case insensitive.

sin(x)	complex sine of x
cos(x)	complex cosine of x
tan(x)	complex tangent of x
asin(x) arcsin(x)	complex arc sine of x
acos(x) arccos(x)	complex arc cosine of x
atan(x) arctan(x)	complex arc tangent of x
atan2(x,y) arctan2(x,y)	arc tangent of y/x, but uses the sign of both arguments to determine the quadrant. Works only with Re{} part of expression

Table 8: Trigonometric functions

sinh(x)	complex hyperbolic sine of x
cosh(x)	complex hyperbolic cosine of x
tanh(x)	complex hyperbolic tangent of x
asinh(x) arcsinh(x)	complex inverse hyperbolic sine of x
acosh(x) arccosh(x)	complex inverse hyperbolic cosine of x
atanh(x) arctanh(x)	complex inverse hyperbolic tangent of x

Table 9: Hyperbolic functions

exp(x)	e (2.71828183...) raised to the power of x
ln(x) log(x)	natural logarithm of x
log10(x)	common logarithm of x
sqrt(x)	square root of x
pow(x,y)	x raised to the power of y

Table 10: Exponential and logarithmic functions

abs(x)	absolute value of x
re(x)	real part of x
im(x)	imaginary part of x
arg(x)	argument of x
conj(x)	complex conjugate
min(x, y)	smallest of x and y NOTE: min(a,b) = min(Re(a), Re(b)) if (Im{a}!=0 || Im{b}!=0)
max(x, y)	biggest of x and y NOTE: max(a,b)= max(Re(a), Re(b)) if (Im{a}!=0 || Im{b}!=0)
deg(x)	for x radians returns number of degrees (0 <= deg(x) < 360) NOTE: deg(x)=deg(Re(x)) if Im{x}!=0
rad(x)	for x degrees returns number of radians (0 <= rad(x) < 2pi) NOTE: rad(x)=rad(Re(x)) if Im{x}!=0
norm(x)	norm of complex number
polar(x)	creates complex number from polar components, where Re(x) = r and Im(x) = t;

Table 11: Miscellaneous functions

5. Constants

There are three predefined constants:

- pi 3.14159265...
- e 2.71828183...
- ans result of previous calculation

Constants are case sensitive.

With complex unit ('i' or 'j') it is allowed to write expressions like this: 2i, 5.5j, etc... With other constants this is not allowed. So, if you want for example double of pi, you must write 2*pi, not 2pi. If you're wondering why is this so, try to interpret this expression: 12e+3. Is it equal to 12*10^3 or is it 12*e+3? So, once again: there is no implicit multiplication with constants. On the other hand, it is allowed to use constant alone instead of multiplying it with one, ie. 34+e, pi+25, (instead of 34+1*e, 1*pi+25) etc...

6. Troubleshoting and FAQ

1. **Why does Launhy become unresponsive while I'm doing calculation?**

   Because you are pushing it to far! :-)

   If you try really big numers, Mathy Resurrected will happily use them in calculations. But, that comes with the cost: speed. This penalty comes mostly from displaying numbers, not calculating them.

   One solution is to consider turning of binary, hexadecimal and octal number base display. If that doesn't work, try setting number display from fixed to scientiffic or automatic. That should do the trick.

   If you really need numbers that big, and really need displaying every single digit of a number, consider using Matlab, Mathematica or some similar solution.

2. **How big numbers are we talking about?**

   It depends on your system. Generaly, everything bigger than 1e1000 will probably slow down things on weaker systems.

   Only for reference, on my system with 2 x 2,0 Ghz processor, anything bigger than 1e100000 will slow things down.

   On the other hand, I believe there is no Launchy skin large enough to display 100000 digits properly :-)

3. **But I've read on other places in this documentation that Mathy Resurrected doesn't even support numbers this big?**

   Well, if you really need to know, it does. You just haven't read docs throughly. On these other places it is said that numbers from certain range can be fully displayed using fixed output format without introducing some small error. Numbers that can be used in calculations are much bigger thatn that, but displaying them using fixed output format introduces small error and is not recomended.

7. Plugin info and history

Mathy resurrected is advanced calculator plugin for Launchy (http://www.launchy.net/). It is authored by Tomislav Adamić (a.k.a. El Gruñón) and hosted on Sourceforge.net:

http://sourceforge.net/projects/mathyresurected/

7.1. History

During the Launchy v1.x.x. era Mathy plugin apeared. It featured everything that users needed and didn't get from Launchy's standard calculator. Now, Launchy being program launcher, nobody probably assumed that this calculator feature would be so wanted or popular. Well, anyone who tries using Launchy with calculator for it's every day job in let's say finance, accounting or programming will witness how usefull this is. And will become addicted to Launchy + Calculator option.

The problem with Launchy plugins is that they need to be rebuilt ever time new Launchy version comes out. This is not really a big problem, as long as plugin users and plugin developers are avare of it existing. And this is where Mathy story ended. Mathy project stopped being developed and when Launchy 2.0 came out, there was no this great calculator that everybody (including me) liked so much...

After Launchy 2.0 there was choice to either use Launchy 1.x with Mathy, or use Launchy 2.0 without your favorite and so much needed plugin. Mathy Resurrected was started to change that fact. The ultimate goal is to produce stable, well featured calculator plugin for Launchy that will replicate almost all features of prevoius plugin Mathy, and also add it's own.

7.2. Features

To the time of writing this (Mathy Resurrected v0.2.0) implementing of all Mathy features is finished. There is always room for improvements, and still more features. As of now these are main things supported:

- Complex number arithmetic – All claculations are done on complex numbers. Where this is inapropriate, it is documented.
- Many built-in math functions.
- Some predefined math constants.
- Configurable decimal separator and function argument separator.
- Syntax for automatic SI units and binary units conversion.
- Input and output of binary, hexadecimal nad octal numbers.
- High precision of calculation supporting calculation and decimal representation of numbers from [1e-70, 1e70], and calculation only (with scientiffic representation) with even greater/smaller numbers.

7.3. Building and installation

Pre-built plugin .dll is available for MS Windows systems, and source code for others.

On windows, installing it means simply copying .dll file into plugins directory of Launchy install dir.

On other systems plugin must be built from source and installed to appropriate place for that system. For detailed instructions see INSTALL in Mathy Resurrected source tree.

7.4. Support

All questions regarding plugin should be asked at SourceForge project site:

http://sourceforge.net/projects/mathyresurrected/

Or at Launchy project site, "Plugins 2.0+" subforum:

http://sourceforge.net/projects/launchy/

Bugs can be reported, or features requested on plugin official project support page:

http://sourceforge.net/projects/mathyresurected/support